

```

l = []
for x in range(10):
    l.append(math.factorial(x)/math.sqrt(x)*math.
log(x))
'''
# timeit execution
print(timeit.timeit(setup=test _ setup,stmt=test _
stmt,number=100000))
>>> 0.009644453999499092.

```

The above snippet of code evaluates the runtime of `example()` routine and outputs the same. It is essential to know that this output is the execution time of 100000 loops of the snippet, for single iteration runtime we must divide the execution time by the `number` parameter.

The `timeit.timeit()` function calls the testing routine only once, which internally loops the code block given '`number`' of times. But the `timeit` module provides another convenience function which repeats the same test multiple times. `repeat()` is the method provided for such instances, which repeatedly calls the `timeit()` function. It has a signature similar to `timeit()` with an added parameter `repeat = n`. It calls the `timeit()` function '`n`' number of times and returns a list of execution results.

```

#repeat execution
print(timeit.repeat(setup=test _ setup,stmt=test _
stmt,repeat=5, number=100000))
[0.009604062001017155, 0.01021480399867869,
0.009828559999732533, 0.011842824998893775,
0.011740521000319859]

```

The command-line interface of the `timeit` module provides almost all the functionalities of the `python` interface, but is invoked from the terminal rather than the interpreter or compiler. It accepts the statement and the parameters as command line arguments of which, the most important ones are listed below along with the syntax:

```
python -m timeit [arguments] [statement]
```

```
-n or --number: number of times to execute the statement
```

```
-r or --repeat: how times it should repeat the timer
```

```
-s or --setup: initial setup statement to be executed once before
```

the test

```
[statement]: the code block under observation
```